

Study Definition Repository (SDR)

Reference Implementation

Solution Architecture

Version 7.0

Disclaimer

These materials and information, as well as the underlying code/application they relate to are provided by TransCelerate Biopharma Inc. AS IS. Any party using or relying on this information and, these materials and/or the underlying code/application do so entirely at their own risk. Neither TransCelerate nor its members will bear any responsibility or liability for any harm, including indirect or consequential harm, that a user may incur from use or misuse of this information, materials, or underlying code/application.

TransCelerate does not endorse any particular software, system, or service. And the use of specific brands of products or services by TransCelerate and its collaboration partners in developing the SDR Reference Implementation should not be viewed as any endorsement of such products or services. To the extent that the SDR Reference Implementation incorporates or relies on any specific branded products or services, this resulted out of the practical necessities associated with making a reference implementation available to demonstrate the SDR's capabilities.

Document History

Version No.	Date	Author	Revision Description
V0.1	Nov 27 th , 2021	Nachiket Vaidya	Initial Version
V1.0	March 11 th , 2022	Nachiket Vaidya	Revised
V2.0	August 19 th , 2022	ACN	Updated diagrams and authentication related content
V3.0	20-Mar-2023	ACN	Updated diagrams and content as per latest architectural changes
V4.0	13-Jul-2023	ACN	Data model and conformance rules links updated to point to USDM V2.0
V5.0	09-Oct-2023	ACN	• Architecture diagram updates ○ Re-arranged components for better clarity. ○ Added API key-based authentication. • Updated content related to API authentication.
V6.0	05-Apr-2023	ACN	Data model and conformance rules links updated to point to USDM V2.0
V7.0	11-Sept-2025	Capgemini	Remove Azure references and add containerization details

Table of Contents

1. Solution Overview	4
1.1. High Level Solution Architecture	5
1.2. Changes for Release V5.0 (September 2025)	6
2. Architecture Goals and Constraints	7
2.1. Architecture Goals/Objectives	7
2.2. Architectural Assumptions and Decisions	7
2.3. Solution Architecture Attributes	8
2.3.1. Tools and Technologies	8
2.3.2. Portability	9
3. Application Architecture	11
3.1. Frontend Application (UI)	11
3.2. API Layer	11
3.3. API Service Specifications	11
3.4. API Architectural Style.....	11
3.5. API Component Model	12
3.6. CDISC Rules Engine.....	12
4. Data Architecture	13
4.1. Conceptual/Logical Data Model	13
4.2. Data Dictionary.....	13

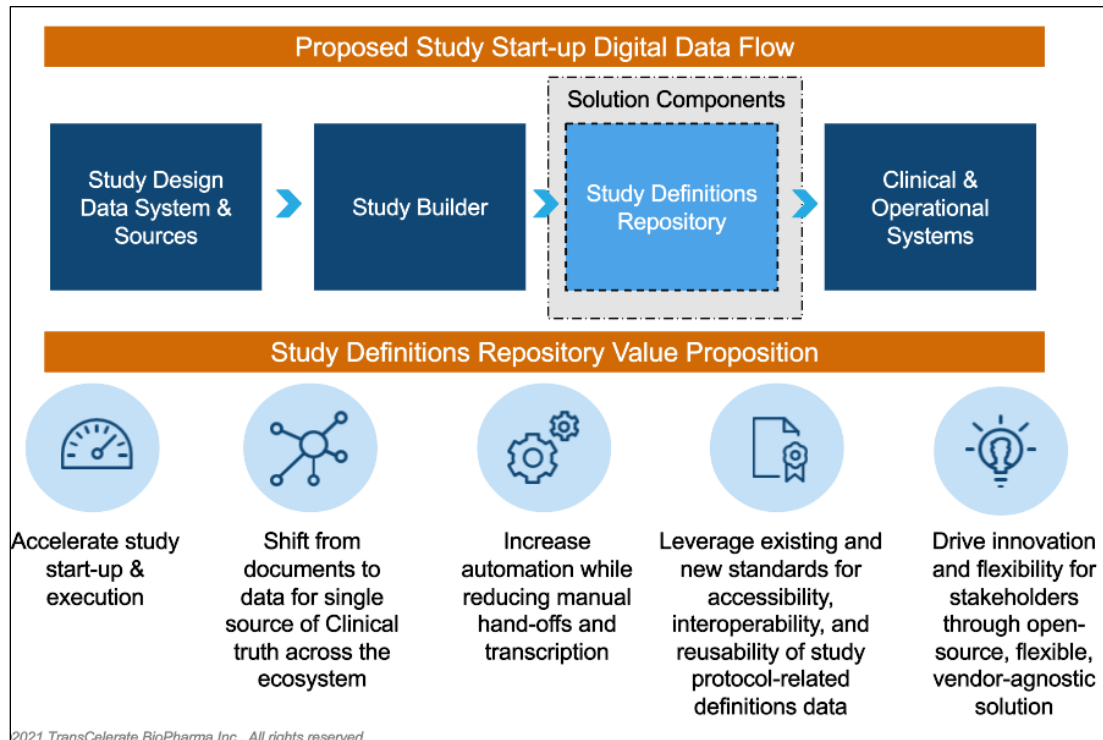
1. Solution Overview

Digital Data Flow (DDF) is TransCelerate's vision to catalyze an industry-level transformation, enabling digital exchange of study definitions (e.g., protocols) by collaborating with standards bodies to create a sustainable open-source Study Definition Repository (SDR) based upon a CDISC standardized model – the Unified Study Definitions Model (USDM). The SDR seeks to support the drug development process by enabling a digital workflow to move from a current state of manual asset creation to a future state of fully automated and dynamic readiness to support clinical study execution.

The SDR Reference Implementation is an attempt to demonstrate the aforesaid ability to support this vision of the DDF initiative and is implemented using Docker¹ containerization architecture for platform independence.

¹ The SDR RI is not a commercial product, rather it is TransCelerate's attempt to illustrate what might be possible in implementing the USDM developed by CDISC. To the extent that the SDR Reference Implementation incorporates or relies on any specific branded products or services, this resulted out of the practical necessities associated with making a reference implementation available to demonstrate the SDR's capabilities. TransCelerate does not endorse any particular software, system, or service. And the use of specific brands of products or services by TransCelerate and its collaboration partners in developing the SDR Reference Implementation should not be viewed as any endorsement of such products or services. Users can use the USDM for any purpose they choose and can build their own implementations of the SDR using the resources available on GitHub.

Figure 1 - Solution Overview Diagram



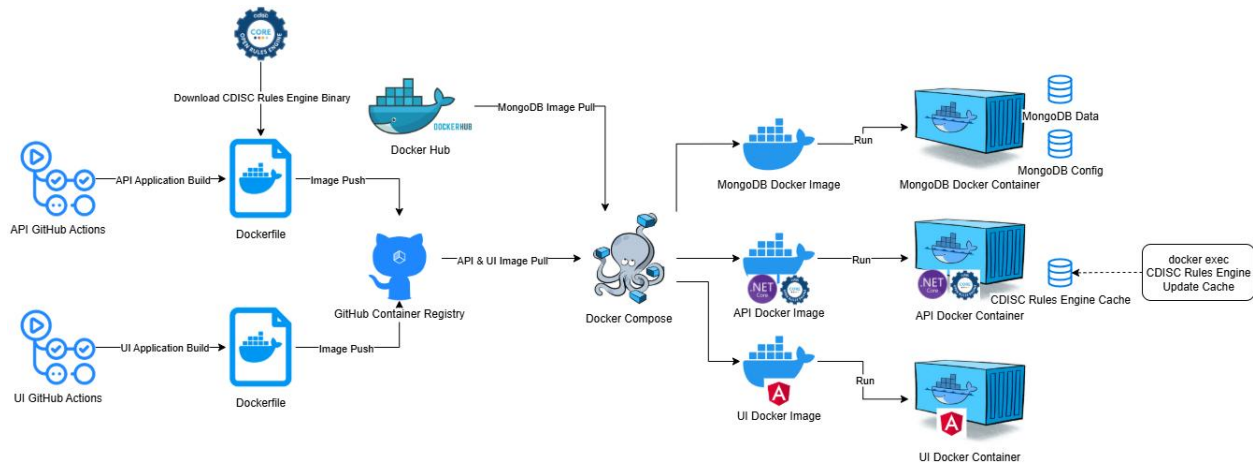
1.1. High Level Solution Architecture

Figure 2 below depicts the high-level architecture of the SDR Reference Implementation, which is built using Angular for a frontend UI, .NET 8 for the backend and containerized using Docker.

The solution architecture components are chosen in a way to support TransCelerate's objectives of making future releases containerized as well as vendor and platform agnostic while ensuring portability and deployment flexibility of the reference implementation across various platforms.

For the Reference Implementation, the vision is to leverage the USDM provided by CDISC by building a repository to house the USDM conformant study data, establishing inbound APIs to enable **upstream systems (e.g., study builders (SB), protocol authors)** to input data, and outbound API to enable outward flow of data to **downstream systems (e.g., Electronic Data Capture systems (EDC))** to automate study start-up activities.

Figure 2 - High Level Solution Architecture Diagram



1.2. Changes for Release V5.0 (September 2025)

SDR Release V5.0 marks a fundamental shift from previous versions by eliminating Azure dependencies from its architecture, more easily enabling platform-agnostic deployment capabilities across various environments.

2. Architecture Goals and Constraints

This section provides a description of the goals and constraints of the Solution Architecture.

2.1. Architecture Goals/Objectives

The architecture for the SDR Reference Implementation has been designed to achieve the following key objectives and architectural goals. The architectural components are chosen in a way to meet the foundational business requirements of being vendor, cloud, and system agnostic.

- **Cloud Agnostic / Open-Source** - Create an application that is relatively platform agnostic from an implementation perspective by choosing a technology stack and containerization components that offer extensibility and portability of the application.
- **Accelerate study start-up / execution** by enabling the automation of data flow to downstream clinical systems which reduces the need for duplication, manual input, and transcription.
- **Reduce Manual input** by creating an application that automates data flow.
- **Open API Specifications** with REST endpoints to maximize system interoperability and promote collaboration.

2.2. Architectural Assumptions and Decisions

The SDR Reference Implementation will leverage Docker containerization components and services for development and deployment. The choice of solution components, however, has been made to achieve key architectural goals and objectives listed in section 2.1 above. The following are some of the key architectural decisions made for the SDR Reference Implementation.

Table 1 - Architectural Decisions

Area	Decision
Connectivity	OAS compliant RESTful API interfaces operate on a Push / Pull model – the upstream vendor is responsible for pushing data into the SDR and the downstream vendor is responsible for pulling data from the SDR. Import and Export

	functionality within the SDR will be considered for future release.
Database	A NoSQL Database (Mongo DB) is used to define the Application Data Model and support data versioning. Additionally, it also disallows the import of data that is not in USDM format.
Authentication and Authorization	Authentication and authorization have been intentionally decoupled from the core application. The containerized architecture provides a foundation for implementers to integrate their own identity management solutions. This architectural decision reduces complexity, enables customization, and allows implementers to leverage existing identity infrastructure.
SDR Upload Version	Study Definitions stored in the repository are assigned a version generated and maintained by the SDR Application.
USDM Version Management	USDM Versions published by CDISC will be maintained by the SDR application for each Study Definition stored in the repository. For a given instance of SDR, it will support the latest 3 major versions of USDM (including all minor versions in between).
Auditing	EntryDatetime and EntrySystem are captured in the Audit Log for SDR Reference Implementation.

2.3. Solution Architecture Attributes

2.3.1. Tools and Technologies

The SDR Reference Implementation is built using Docker containerization technology, with application components designed to ensure portability and interoperability across different systems or environments. This container-based approach eliminates

dependencies on specific cloud providers such as Microsoft Azure², Amazon Web Services (AWS)², or Google Cloud Platform (GCP)².

Below is the list of tools and technologies adopted in designing and developing the DDF SDR Reference Implementation.

Table 2 - Tools and Technologies

Front End UI Application	Backend API Application	DevOps	Testing
Angular 13 Bootstrap HTML5	.NET 8	Docker GitHub	• NUnit Testing • Postman for API Testing • SDR UI - Manual Testing

2.3.2. Portability

The architecture of the SDR Reference Implementation has been designed with portability in mind, which is defined as the ability to move and suitably adapt the applications and data between different environments. Docker containerization enables deployment across various platforms without modification. Specifically:

Data Portability: The data model of the solution has been designed using Mongo DB, which stores data in collections in JSON format. The container-based database approach allows for easy transfer of data between environments without dependency on cloud-specific services. The standardized JSON document format ensures compatibility with any NoSQL database.

Application Portability: The technology stack chosen for the Reference Implementation ensures ease of deploying the application components to different environments through Docker containerization. The SDR RI codebase does not require

² To be clear, TransCelerate does not endorse any particular software, system, or service. And the use of specific brands of products or services by TransCelerate and its collaboration partners in developing the SDR Reference Implementation should not be viewed as any endorsement of such products or services. To the extent that the SDR Reference Implementation incorporates or relies on any specific branded products or services, this resulted out of the practical necessities associated with making a reference implementation available to demonstrate the SDR's capabilities.

changes when moving between platforms, as containers encapsulate all dependencies. The front-end of the application is built using Angular and is containerized. The front-end container image is available on GitHub Container Registry (GHCR). The API layer is built using .NET 8 and runs in its own Docker container. The CDISC Rules Engine binary used for USDM V4.0 conformance validation runs alongside the .NET 8 API inside the API Docker container. The API container image is also available on GHCR.

3. Application Architecture

The application architecture section describes and defines the solution's architecture, including the major solution components, their relationships and the technologies and tools used to build them. The Application Layer for the Solution has a frontend (UI) Application built using Angular 13 and an API Layer built using .NET 8 and follows the Open API Specifications.

3.1. Frontend Application (UI)

The frontend application for the solution is designed using a Single Page Application (SPA) pattern and follows a Model View Controller (MVC) architectural style to integrate with the backend solution components. The User Interface for the application has been designed to enable the user to perform several business functions such as View Study Details, Search Study Definitions, Study Comparison and System Usage Report.

3.2. API Layer

The API layer of the SDR Reference Implementation is designed to expose endpoints to be called externally as well as consumed by the SDR UI. The SDR V5 API is integrated with the CDISC Rules Engine to validate USDM V4.0 conformance.

3.3. API Service Specifications

The API Layer of the solution complies with the OpenAPI Specification (OAS) standards which allows systems to discover and understand the capabilities of the service without access to the source code, documentation, or through the network traffic inspection. When properly defined, a consumer can understand and interact with remote services with a minimal amount of implementation logic.

3.4. API Architectural Style

The API Layer for the SDR follows the REST architectural style that uses HTTP requests to GET, PUT, and POST data. REST standards are not linked with any technology or platform, and introduce the best practices known as constraints. They also describe how the server processes requests and responds to them. Operating within RESTful architecture constraints, the system gains desirable properties such as reliability, ease of use, improved scalability and security, low latency while enhancing the system performance and helping achieve technology independence in the process.

3.5. API Component Model

The API Layer components for the SDR Reference Implementation are developed using .NET classes written in C#.

3.6. CDISC Rules Engine

The API runs the CDISC Rules Engine binary as a process for each USDM V4.0 conformance validation. The USDM rules persist in a volume and can be updated from the CDISC library using docker exec to run the rules resource cache update.

4. Data Architecture

Data architecture is dependent on multiple factors mainly associated with how data moves across different upstream and downstream systems. Specifically, to standardize the data exchange between different systems, the SDR Reference Implementation only persists the data in a USDM conformant - JSON format in Mongo DB. The SDR RI leverages one exclusive collection for storing study definitions while supporting features like change audit are maintained in separate collections. The study definition collection is a common collection that stores all study definitions irrespective of the USDM version they conform to.

The NoSQL database design allows for persisted Study Definitions as JSON documents with built in support for tracking changes and auto indexing to ensure optimum system performance. Additionally, it also supports future SDR needs related to data partitioning.

4.1. Conceptual/Logical Data Model

Refer to the CDISC USDM V4.0 data model (https://github.com/cdisc-org/DDF-RA/blob/v4.0.0/Deliverables/UML/USDM_UML.png).

4.2. Data Dictionary

The Data Dictionary captures the relationships for all the data elements of Study Definition. For more details, refer CDISC portal (<https://www.cdisc.org/ddf>).

Appendix A - Key Terms

Table below provides definitions and explanations for terms and acronyms relevant to the content presented within this document.

Table 3 - Appendix A: Key Terms

Term	Definition
API	Application Programming Interface
AWS	Amazon Web Services
DDF	Digital Data Flow
EDC	Electronic Data Capture
GCP	Google Cloud Platform
RI	Reference Implementation
SB	Study Builder
SDR	Study Definition Repository
USDM	Unified Study Definition Model
GHCR	GitHub Container Registry